

# Documentación: Patrón MVC + Observer

*Space Invaders — Proyecto de Programación*

Fecha: 17 de marzo de 2026

---

## Índice

---

|                                                                                     |          |
|-------------------------------------------------------------------------------------|----------|
| <b>1. Introducción al Patrón MVC + Observer</b>                                     | <b>2</b> |
| <b>2. Implementación en Space Invaders</b>                                          | <b>2</b> |
| 2.1. Arquitectura MVC del proyecto . . . . .                                        | 2        |
| 2.2. Diagramas UML . . . . .                                                        | 5        |
| <b>3. El Controlador en Detalle</b>                                                 | <b>5</b> |
| <b>4. Patrón Observer: Notificación y Actualización</b>                             | <b>6</b> |
| 4.1. Método <code>notify</code> . . . . .                                           | 6        |
| 4.2. Método <code>update</code> . . . . .                                           | 7        |
| 4.3. Funcionamiento conjunto de <code>notify</code> y <code>update</code> . . . . . | 7        |
| 4.4. Aplicación al proyecto Space Invaders . . . . .                                | 8        |
| 4.4.1. Quién notifica y cuándo . . . . .                                            | 8        |
| 4.4.2. Estructura de datos en las notificaciones . . . . .                          | 9        |
| <b>5. Conclusiones</b>                                                              | <b>9</b> |

## Introducción al Patrón MVC + Observer

---

El patrón **Modelo-Vista-Controlador (MVC)** es una arquitectura de diseño utilizada en programación orientada a objetos que permite organizar una aplicación separando sus componentes en tres partes principales: *Modelo*, *Vista* y *Controlador*. Esta separación facilita el mantenimiento del código, mejora su organización y permite que cada componente tenga una responsabilidad clara dentro del sistema.

### Modelo

Se encarga de gestionar los datos y la lógica del programa. Contiene toda la lógica de negocio, del juego en este caso, y las operaciones necesarias para modificarla o consultarla. El modelo no depende de la interfaz gráfica ni de cómo se muestran los datos al usuario.

### Vista

Es la parte encargada de presentar la información al usuario. Su función principal es mostrar los datos que proporciona el modelo y actualizarse cuando estos cambian. La vista no contiene la lógica principal del programa, sino únicamente la representación visual de la información.

### Controlador

Actúa como intermediario entre el usuario y el sistema. Recibe las acciones del usuario (por ejemplo, clics, entradas de teclado, etc.), interpreta estas acciones y solicita al modelo que realice las operaciones correspondientes.

A este modelo de arquitectura se le añade el patrón **Observer (Observador)** para que la vista se mantenga actualizada cuando cambian los datos del modelo. Este **patrón de diseño** establece una relación entre un objeto denominado *observable* (el modelo) y uno o varios *observadores* (las vistas). Cuando el estado del observable cambia, todos los observadores registrados son notificados automáticamente y pueden actualizar su información.

La combinación de MVC y Observer permite desarrollar aplicaciones más modulares, donde la lógica del programa, la presentación de los datos y la interacción con el usuario están claramente separadas. Esto facilita la reutilización del código, la ampliación del sistema y el mantenimiento a largo plazo de la aplicación.

## Implementación en Space Invaders

---

### Arquitectura MVC del proyecto

En el proyecto Space Invaders, las tres capas del patrón MVC se distribuyen de la siguiente manera:

### Modelo

La clase central del modelo es **Espacio**, que está implementado como **Singleton** y extiende de la clase **Observable**. Contiene toda la lógica del juego: la instancia del **Jugador**, la lista de **Enemigos** y el objeto **Disparo** asociado al jugador. Además, **Espacio** alberga el bucle principal del juego (*game loop* con un **Timer** de 50 ms), la detección de colisiones entre el disparo y los enemigos, y las condiciones de fin de partida (*game over* y victoria).

Las clases auxiliares del modelo definen la estructura de las naves y los disparos del juego:

- **Naves** — clase abstracta que encapsula el comportamiento común de todas las naves. Define atributos: **x** (posición horizontal), **y** (posición vertical), **velocidad** (píxeles por actualización, por defecto 1) y **vivo** (estado actual). Incluye el método abstracto **mover()**, que cada subclase implementa según su lógica de movimiento. **Nota:** El atributo **velocidad** solo lo utiliza la clase **Jugador**; la clase **Enemigo** no lo usa.
- **Jugador** — extiende **Naves** y representa la nave controlable. Implementa **mover()** multiplicando los desplazamientos por **velocidad**, permitiendo movimiento dentro de los límites del tablero visible (0-99 en x, 0-59 en y). Posee un objeto **Disparo** que se dispara cuando el usuario presiona espacio. Solo hay una instancia del jugador simultáneamente y solo un disparo activo por jugador.
- **Enemigo** — extiende **Naves** y representa un enemigo del juego. Implementa **mover()** sumando directamente los desplazamientos sin usar **velocidad**. Se crean inicialmente en una posición aleatoria en la fila superior (y entre 0-4) y descienden verticalmente. Se eliminan al salir del tablero o ser impactados por un disparo. El juego comienza con un único enemigo.
- **Disparo** — representa el proyectil disparado por el jugador. Almacena su posición actual (**x**, **y**) y un booleano (**activo**) para saber si está en vuelo. Se crea un nuevo **Disparo** cada vez que el jugador presiona espacio, con posición inicial (**x\_jugador**, **y\_jugador - 1**), reemplazando el anterior. Ascende un píxel por tick (cada 50ms) mediante el método **subir()**. Se desactiva automáticamente cuando alcanza **y<0** (sale del tablero).

Ninguna de estas clases conoce la interfaz gráfica ni sabe cómo se dibuja el juego; solo gestionan datos y reglas de juego.

## Vista

Las vistas son **StartFrame** y **MainFrame**, ambas subclases de **JFrame** e implementan la interfaz **Observer**. Su función es capturar el estado del modelo y representarlo visualmente sin contener lógica de negocio:

- **StartFrame** es la pantalla inicial del juego. Se encarga de mostrar el título ("SPACE INVADERS"), el subtítulo ("Sprint 1"), las instrucciones básicas de control, y el mensaje «Pulsa SPACE para jugar». No dibuja ningún elemento del juego ni gestiona la lógica de juego. Su responsabilidad es permitir que el usuario active el inicio de la partida.
- **MainFrame** es la pantalla de juego principal y gestiona toda la representación gráfica. Usa una matriz de **JLabels** para representar el tablero como cuadrícula negra de  $100 \times 60$  celdas (cada celda de  $10 \times 10$  píxeles). Los elementos se representan con colores: el jugador como celdas magenta, los enemigos como celdas rojas, y el disparo como celda amarilla. También muestra un mensaje de victoria o derrota cuando la partida termina.

El mecanismo clave es que ambas vistas se registran automáticamente como observadoras de **Espacio** al construirse, mediante la llamada **Espacio.getEspacio().addObserver(this)** en sus constructores. Esto garantiza que recibirán notificaciones automáticas de cualquier cambio en el estado del juego.

## Controlador

Los controladores son clases internas privadas llamadas **Controller** definidas dentro de **StartFrame** y **MainFrame**. Ambas implementan **KeyListener**, lo que requiere implementar obligatoriamente los tres métodos de la interfaz: **keyPressed**, **keyReleased** y **keyTyped**. En la práctica, solo **keyPressed** contiene lógica (los otros dos están vacíos). Esto permite que el controlador reciba los eventos del teclado, interprete las teclas pulsadas y traduzca en llamadas concretas al modelo:

- El **Controller** de **StartFrame** escucha la pulsación de **SPACE** para llamar a `Espacio.getEspacio().cambiarAMain()`, indicando al modelo que inicie la partida.
- El **Controller** de **MainFrame** atiende las cuatro flechas direccionales (llamando a `espacio.moverJugador(dx, dy)` con los desplazamientos) y la barra espaciadora (llamando a `espacio.disparar()`). La verificación de si la partida ha terminado **no está en el Controller**, sino en el **modelo** (en los métodos `moverJugador()` y `disparar()`).

La relación entre las tres capas queda así: el usuario interactúa con la **Vista** (ventana con foco de teclado), el **Controlador** captura esa interacción y llama al **Modelo**, y el Modelo notifica a la Vista para que se hagan los cambios necesarios. En ningún momento la vista accede directamente a la lógica del juego, ni el modelo sabe cómo se representa gráficamente.

Más adelante se verá como se implementa esta relación entre las tres capas en el proyecto Space Invaders.

## Diagramas UML

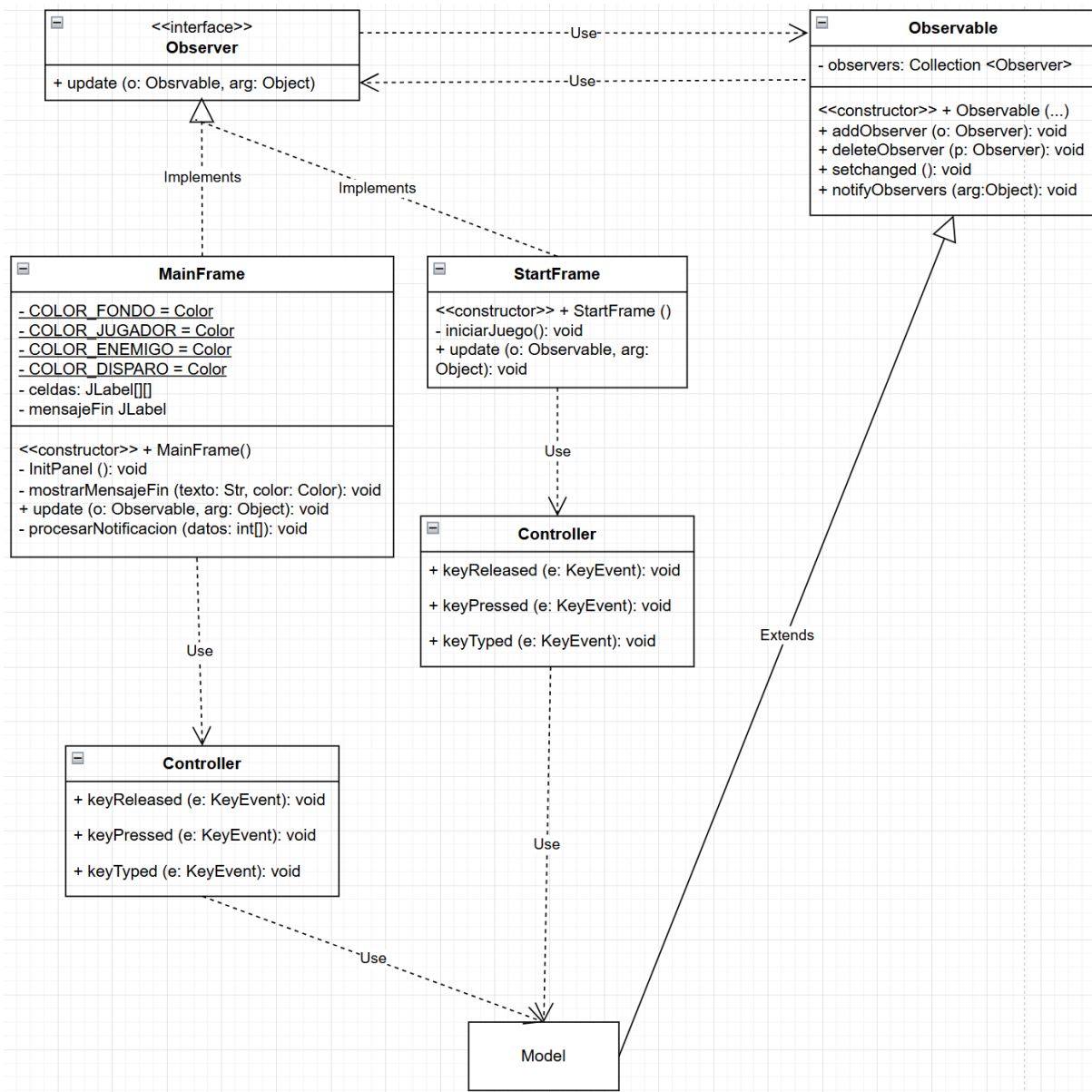


Figura 1: Diagrama UML de la arquitectura MVC y relación Observer entre las clases del proyecto Space Invaders.

## El Controlador en Detalle

El controlador es el intermediario entre el usuario y el sistema. Recibe las acciones del usuario (eventos de teclado), las interpreta según la lógica de control definida, y solicita al modelo que execute las operaciones correspondientes. No contiene datos del juego ni se encarga de mostrarlos; simplemente traduce entrada en acciones del modelo.

En ambas clases (**StartFrame** y **MainFrame**), los controladores están definidos como clases internas privadas llamadas **Controller** que implementan **KeyListener**. Esto requiere implementar obligatoriamente los tres métodos de la interfaz: **keyPressed**,

`keyReleased` y `keyTyped`. En la práctica, solo `keyPressed` contiene lógica (los otros dos están vacíos).

El flujo común de funcionamiento es el siguiente:

1. En el constructor de cada frame se crea una instancia del controlador e inmediatamente se registra con `addKeyListener(controller)`.
2. La ventana solicita el foco del sistema con `requestFocusInWindow()`. Esto es crucial porque solo la ventana que tiene el foco puede recibir eventos del teclado.
3. Cuando el usuario pulsa una tecla, el sistema operativo detecta el evento y lo envía automáticamente al método `keyPressed` del `Controller` registrado. Este método recibe un objeto `KeyEvent` que contiene información sobre qué tecla fue pulsada.

### En la clase `StartFrame`

En esta clase, su única función es detectar si el usuario pulsa la barra espaciadora, en tal caso llama a `iniciarJuego()` que veremos posteriormente qué hace.

### Clase `MainFrame`

El `Controller` de esta clase traduce las teclas del usuario en acciones del juego. En `keyPressed` se implementa la lógica específica:

- Lee qué tecla se pulsó con `e.getKeyCode()`.
- Según la tecla, llama al modelo `Espacio`:
  - **Flechas direccionales:** ejecutan `espacio.moverJugador(dx, dy)` con los desplazamientos correspondientes (izquierda: -1,0; derecha: 1,0; arriba: 0,-1; abajo: 0,1).
  - **Espacio:** ejecuta `espacio.disparar()`.

**IMPORTANTE:** El `Controller` **no verifica** si la partida ha terminado. Esa validación ocurre dentro del modelo en los métodos `moverJugador()` y `disparar()`, que comprueban internamente con `isGameOver()` y `isGameWon()` antes de ejecutar. Este fue uno de los muchos cambios que se hicieron tras el feedback del profesor.

## Patrón Observer: Notificación y Actualización

---

Dentro del patrón Observer, los métodos `notify` y `update` permiten que los objetos que observan a otro objeto puedan mantenerse informados cuando su estado cambia.

### Método `notify`

El método `notify` pertenece al objeto **observable**, que es el objeto que contiene los datos o el estado que puede cambiar. En muchas aplicaciones que siguen la arquitectura MVC, el observable suele ser el modelo, ya que es el componente que gestiona la información y la lógica del sistema.

La función del método `notify` es avisar a todos los observadores registrados de que el estado del observable ha cambiado. Para ello, el observable mantiene normalmente

una lista o colección de observadores que se han registrado previamente para recibir notificaciones.

Cuando ocurre un cambio relevante en el estado del observable (por ejemplo, una modificación de datos), el método `notify` recorre esta lista de observadores y llama al método `update` de cada uno de ellos. De esta manera, todos los observadores reciben la notificación del cambio y pueden reaccionar en consecuencia.

Una característica importante del método `notify` es que el observable **no necesita conocer los detalles internos de los observadores**. Simplemente se encarga de avisarlos de que ha ocurrido un cambio. Esto permite mantener un bajo acoplamiento entre las clases, lo que facilita la reutilización del código y el mantenimiento del sistema.

## Método `update`

El método `update` pertenece a los **observadores**. Cada observador debe implementar este método para definir cómo reaccionará cuando el observable le notifique un cambio.

Cuando el observable ejecuta el método `notify`, este llama al método `update` de cada observador registrado. En ese momento, el observador recibe la notificación y puede realizar las acciones necesarias para actualizar su estado.

En muchos casos, el método `update` se utiliza para que el observador consulte el nuevo estado del observable y adapte su comportamiento o su representación visual. Por ejemplo, si el observable es el modelo de una aplicación y el observador es una vista, el método `update` se encargará de actualizar la información que se muestra al usuario para reflejar los cambios recientes.

Cada observador puede implementar su propio comportamiento en el método `update`, lo que permite que diferentes componentes reaccionen de distintas formas ante el mismo cambio en el observable.

## Funcionamiento conjunto de `notify` y `update`

El funcionamiento coordinado de estos dos métodos permite mantener sincronizados los distintos componentes de un sistema que utiliza el patrón Observer. El proceso general es el siguiente:

1. Se produce un cambio en el estado del objeto observable.
2. El observable detecta ese cambio y ejecuta el método `notify`.
3. El método `notify` recorre la lista de observadores registrados.
4. Para cada observador, se llama a su método `update`.
5. Cada observador recibe la notificación y actualiza su estado o su información según corresponda.

Gracias a este mecanismo, los cambios en el sistema se propagan automáticamente a los objetos que dependen de ellos, sin necesidad de que exista una dependencia directa entre todas las clases.

## Aplicación al proyecto Space Invaders

### 4.4.1. Quién notifica y cuándo

En el proyecto, el observable es `Espacio`. Las notificaciones a los observadores se realizan directamente con la secuencia:

```
setChanged();  
notifyObservers(new int[] {tipo, datos...});
```

La llamada a `setChanged()` es obligatoria antes de `notifyObservers()`: sin ella, la clase `Observable` no propagaría nada. `notifyObservers()` recorre internamente la lista de observers registrados y llama al `update` de cada uno, pasando un array de enteros con información específica sobre el cambio.

Las notificaciones se invocan desde varios puntos del modelo, cada vez que el estado del juego cambia y la pantalla debe reflejarlo. Los tipos de notificación (primer elemento del array) son:

- **Tipo 0: Movimiento del jugador** — dentro de `moverJugador()`, tras actualizar la posición. Array: {0, `oldX`, `oldY`, `newX`, `newY`}
- **Tipo 1: Nuevo disparo** — dentro de `disparar()`, tras crear un nuevo `Disparo`. Array: {1, `newX`, `newY`}
- **Tipo 2: Movimiento del disparo** — dentro de `actualizarDisparo()`, en cada tick del *game loop* (cada 50 ms) mientras el proyectil está activo. Array: {2, `oldX`, `oldY`, `newX`, `newY`}
- **Tipo 3: Disparo sale del tablero** — dentro de `actualizarDisparo()`, cuando el disparo alcanza `y=0`. Array: {3, `oldX`, `oldY`}
- **Tipo 4: Movimiento de enemigo** — dentro de `actualizarEnemigos()`, cada 200 ms (cada 4 ticks de 50ms), tras desplazar un enemigo. Array: {4, `oldX`, `oldY`, `newX`, `newY`}
- **Tipo 5: Colisión disparo-enemigo** — dentro de `comprobarColisiones()`, cuando el disparo alcanza a un enemigo. Array: {5, `disparoX`, `disparoY`, `enemigoX`, `enemigoY`}
- **Tipo 6: Inicialización del juego** — dentro de `notificarInicializacion()`, al abrir `MainFrame`. Pinta el jugador y el **primer** enemigo del array. Array: {6, `jugadorX`, `jugadorY`, `enemigoX`, `enemigoY`}
- **Tipo 7: Game Over** — dentro de `actualizarEnemigos()`, cuando se detecta que un enemigo ha llegado a la fila del jugador. Array: {7}
- **Tipo 8: Victoria** — dentro de `comprobarColisiones()`, cuando se elimina el último enemigo vivo. Array: {8}
- **Tipo 9: Cambio a MainFrame** — dentro de `cambiarAMain()`, para señalar que `StartFrame` debe ceder control a `MainFrame`. Solo es procesado por `StartFrame`, no por `MainFrame`. Array: {9}



#### 4.4.2. Estructura de datos en las notificaciones

Cada notificación es un array de enteros que comienza con un identificador de tipo (0-9) seguido de parámetros específicos:

##### Primer elemento (índice 0)

Siempre es el tipo de notificación (0-9). Define qué acción ocurrió en el modelo.

**IMPORTANTE:** El hecho de que exista un identificador de tipo en la vista, y que esta distinga entre varias acciones, no implica que la vista implemente lógica de negocio. La vista simplemente recibe el array de datos, y según el caso que se esté dando, se pinta la celda o celdas del color correspondiente.

##### Elementos posteriores

Dependen del tipo. Pueden ser coordenadas (x, y), posiciones anteriores y nuevas, o una combinación de ambas. Como la vista conoce el tipo de notificación a procesar, no surgirán errores de índice.

**Ejemplo práctico:** Cuando el jugador se mueve de (50, 55) a (51, 55) presionando flecha derecha, el modelo ejecuta:

```
setChanged();  
notifyObservers(new int[] {0, 50, 55, 51, 55});
```

Esta notificación es recibida por el Observer (**MainFrame**), que la procesará y pintará la celda (50, 55) de color negro y la celda (51, 55) de color magenta.

## Conclusiones

---

La implementación del patrón MVC junto con Observer en el proyecto Space Invaders demuestra las ventajas de una arquitectura bien estructurada:

- **Separación clara de responsabilidades:** El modelo (**Espacio**) gestiona exclusivamente la lógica del juego, las vistas (**StartFrame**, **MainFrame**) se encargan únicamente de la presentación, y los controladores traducen la interacción del usuario en acciones del modelo.
- **Bajo acoplamiento:** Las clases no dependen directamente unas de otras. El modelo no conoce las vistas, y las vistas no contienen lógica de negocio.
- **Sincronización automática:** El patrón Observer garantiza que cualquier cambio en el modelo se refleje inmediatamente en todas las vistas registradas, sin necesidad de llamadas explícitas.
- **Facilidad de mantenimiento:** Los cambios en la lógica del juego solo afectan al modelo, los cambios en la interfaz solo afectan a las vistas, y las modificaciones en los controles solo afectan a los controladores.
- **Extensibilidad:** Es fácil añadir nuevas vistas (por ejemplo, una pantalla de puntuaciones) o nuevos tipos de interacción sin modificar el código existente.

Esta arquitectura proporciona una base sólida para el desarrollo de aplicaciones interactivas, facilitando tanto el desarrollo inicial como el mantenimiento y la evolución futura del software.